

Persistent Age/Sex Profile for Triage

SehatAI — Symptom Triage & Session Architecture • Implementation Plan

Context

The symptom-triage bot never asks for age/sex in conversation. Every turn it silently pulls them from Supabase (`Patientprofile.js` → `patients.date_of_birth/sex`), and when that row has no data — true for CareLink-bridged patients today, since the bridge only ever writes name + consented_at — it falls back to a hardcoded **age 30, sex male** (`processMessage.js:1951-1952`) with nothing but a server console warning. The patient is never told a recommendation was computed against fabricated demographics, and there is no UI anywhere (chat or CareLink) that lets them see or change what's on file.

This plan also documents how sessions/fetching currently work, and what (if anything) needs to change for the diet bot to keep working smoothly alongside this change. It is scoped to fit into the UI already built (the two separate Symptoms / Diet pages) — no UI restructuring here.

How Sessions & Fetching Work Today (no change)

- **Single store**, `chatLog.js`. Hybrid RAM + Supabase: an in-memory Map (`sessionExtraStore`) holds live conversation bookkeeping (accumulated symptoms, pending-question flags, clarification counters), backed by a `chat_session_state` Supabase row so a restart doesn't lose an in-progress chat. The RAM cache is swept after 30 min idle — unrelated to session expiry, just a memory-cache eviction; a cache miss re-hydrates from Supabase.
- **Session identity is server-resolved, never client-supplied.** `webserver.js`'s `/api/chat` authenticates the bearer token → `patientId`, then resumes or creates a session server-side. The client only controls a `newSession` boolean; a client-sent session id is never read.
- **24-hour TTL, checked lazily.** Each read compares “now” against the session's last-updated timestamp — no cron sweep needed.
- **Two modes, two pointer tables, one mechanism.** `active_session_pointers` (symptom) and `diet_session_pointers` (diet) each map patient → session independently, so a patient can have a live symptom session and a live diet session at the same time without collision.
- **No raw transcript is ever persisted.** `chatLog.js`'s message-logging function is an intentional no-op (a compliance decision) — only derived state is stored, never the patient's literal sentences. This is why age/sex must live in a real profile table rather than be inferred from a message log.
- **Age/sex are patient-scoped, not session-scoped.** They live on the `patients` row, fetched fresh every turn — so once set correctly, they automatically apply to every future session with zero extra plumbing.

What's Being Added

1. A real, persistent, user-editable profile field for DOB + sex

CareLink and SehatAI now share one Postgres instance, so this is a same-database write, not a cross-service API call.

- `backend/app/models.py` — add `date_of_birth` (Date) and `sex` (String, 'male'/'female') columns to User. CareLink has no migration tool (plain `create_all`), so a one-time manual `ALTER TABLE` is needed against the shared Supabase Postgres.
- `backend/app/schemas.py` — add both fields (optional) to `ProfileUpdate` and `UserOut`. Validate `sex` against a strict male/female set and `date_of_birth` as not-in-the-future / not implausibly old.
- `backend/app/routers/users.py` — `PATCH /users/me` writes the new fields when present, same pattern as the existing name/specialization handling.
- **Sync into SehatAI's own patients row** (which already has `date_of_birth/sex` columns, currently populated only by a seed script): a small helper in `dependencies.py`, called both at bridge-creation time and after every profile update, so a change is live for the patient's very next chat message.
- **Frontend** — extend the existing Profile page (patient-only branch): add a date input for DOB and a Male/Female select (binary only, matching Infermedica's actual API constraint). Same save flow as the current name field — no new page, no change to the Symptoms/Diet chat pages themselves.

2. Stop defaulting silently — gate triage on a complete profile

`processMessage.js`, Stage 5 (≈ line 1937) — replace the silent age-30/male fallback with a hard gate: if DOB or sex is missing, short-circuit with a new response kind:

```
return envelope({
  kind: 'profile_incomplete',
  sessionId,
  reply: "Before I can give you a specialist recommendation, I need "
    + "your age and sex on file – they affect which conditions are "
    + "actually likely. Please complete your profile, then come back "
    + "and we'll pick this up.",
  actionable: true,
  actions: [{ id: 'complete_profile', label: 'Complete your profile' }],
});
```

- This sits **after** the existing emergency / off-topic / dependent gates, so a real emergency is never blocked by an incomplete profile — only the ordinary symptom-gathering path is.
- Frontend — the existing generic action-button renderer (already built for the emergency Continue/Notify buttons) gains one more case: action id `complete_profile` navigates to the Profile page instead of resending a message. No new UI chrome.
- **No backfill** for existing/seeded accounts — the gate applies immediately. Any account being demoed needs its profile completed beforehand, or the first symptom message will hit the new gate.

3. Disclose that the recommendation used age/sex

`processMessage.js`'s `envelope()` function (≈ line 147) is the single choke point every response passes through. When the response kind is 'recommendation' or 'emergency' — the only two kinds where Infermedica was actually called — and age/sex were resolved, append one deterministic sentence to the reply, e.g.:

```
" (This reflects your profile: age {age}, {sex}.)"
```

Added once, at finalization only — not on every clarifying turn — so it stays conversational rather than reciting a form field back every message.

4. Diet bot — no functional changes required

- DietBot doesn't use age/sex today and has its own separate session store, so none of the above touches it.
- Both modes' pointer tables already key off the same SehatAI `patients.id` row created by the one shared bridge — so the moment DOB/sex are synced there, that data is already sitting on the row DietBot's pointer references, with zero extra plumbing, if diet personalization ever wants it later.

Files Touched

File	Change
backend/app/models.py	User gains <code>date_of_birth</code> , <code>sex</code> columns
backend/app/schemas.py	<code>ProfileUpdate</code> , <code>UserOut</code> gain the two fields
backend/app/routers/users.py	<code>update_my_profile</code> writes the new fields
backend/app/dependencies.py	New <code>sync_sehatai_profile</code> helper, called from the bridge and the profile-update route
frontend/templates/index.html frontend/static/js/app.js	Profile page: DOB + sex fields (patient-only) Chat panel: new <code>complete_profile</code> action case
processMessage.js	Stage 5 profile-completeness gate (~1937) <code>envelope()</code> disclosure sentence (~147)
(ops, one-time)	<code>ALTER TABLE users ADD COLUMN date_of_birth date,</code> <code>ADD COLUMN sex varchar;</code> — shared Supabase Postgres

Verification

- Fresh account, no profile data: send a symptom message → confirm a `profile_incomplete` reply + “Complete your profile” button appears (not a silent age-30/male guess), and clicking it lands on the Profile page.
- Fill in DOB + sex on the Profile page, save, return to Symptoms, resend the same message → confirm it now proceeds normally.
- Walk a session through to an actual recommendation → confirm the reply ends with the age/sex disclosure sentence, and confirm a mid-conversation clarification turn does *not* carry that sentence (only the final one does).
- Trigger a real emergency phrase on a profile-incomplete account → confirm the emergency reply still fires immediately, unblocked by the new gate.
- Switch to the Diet page on the same account → confirm it's unaffected (no gate, no disclosure, sends/receives normally) — diet mode never calls Stage 5.

- `node testEdgeCases.js` — full regression pass.